0143. Reorder List

y: Medium | Category: Linked List | LeetCode: https://leetcode.com/problems/reo

Problem

- Reorder L0→L1→…→Ln into L0→Ln→L1→Ln-1→… in place.
- Example: 1,2,3,4,5 → 1,5,2,4,3.

Intuition

- The result alternates between the front half and the reversed back half.
- So: find the middle, reverse the second half, then weave the two lists together.

Brute Force

- Put nodes in an array and relink with two indices: O(n) extra space.

Optimal Approach

- Slow/fast pointers find the middle; cut after slow.
- Reverse the second half (problem 206).
- Merge alternately: a1, b1, a2, b2, ...

Worked Example

```
1->2->3->4->5.

middle at 3 -> first 1,2,3   second 4,5
reverse second -> 5,4
weave: 1,5,2,4,3
```

Java Solution

```java
public class ReorderList {
    public void reorderList(ListNode head) {
        if (head == null) return;
        ListNode slow = head, fast = head.next;
        while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
        ListNode second = slow.next, prev = null;
        slow.next = null;
        while (second != null) { ListNode nx = second.next; second.next = prev; prev = second; second = nx; }
        ListNode first = head;
        second = prev;
        while (second != null) {
            ListNode n1 = first.next, n2 = second.next;
            first.next = second;
            second.next = n1;
            first = n1;
            second = n2;
        }
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- Cut the list (slow.next = null) or you create a cycle.